

EEL670 - Linguagens de Programação

ACERVO DE LIVROS

Rafael Gabriel - 121087555

Luiza Pacheco - 119169147

SUMÁRIO

1. Problema

2. Solução

3. Arquitetura

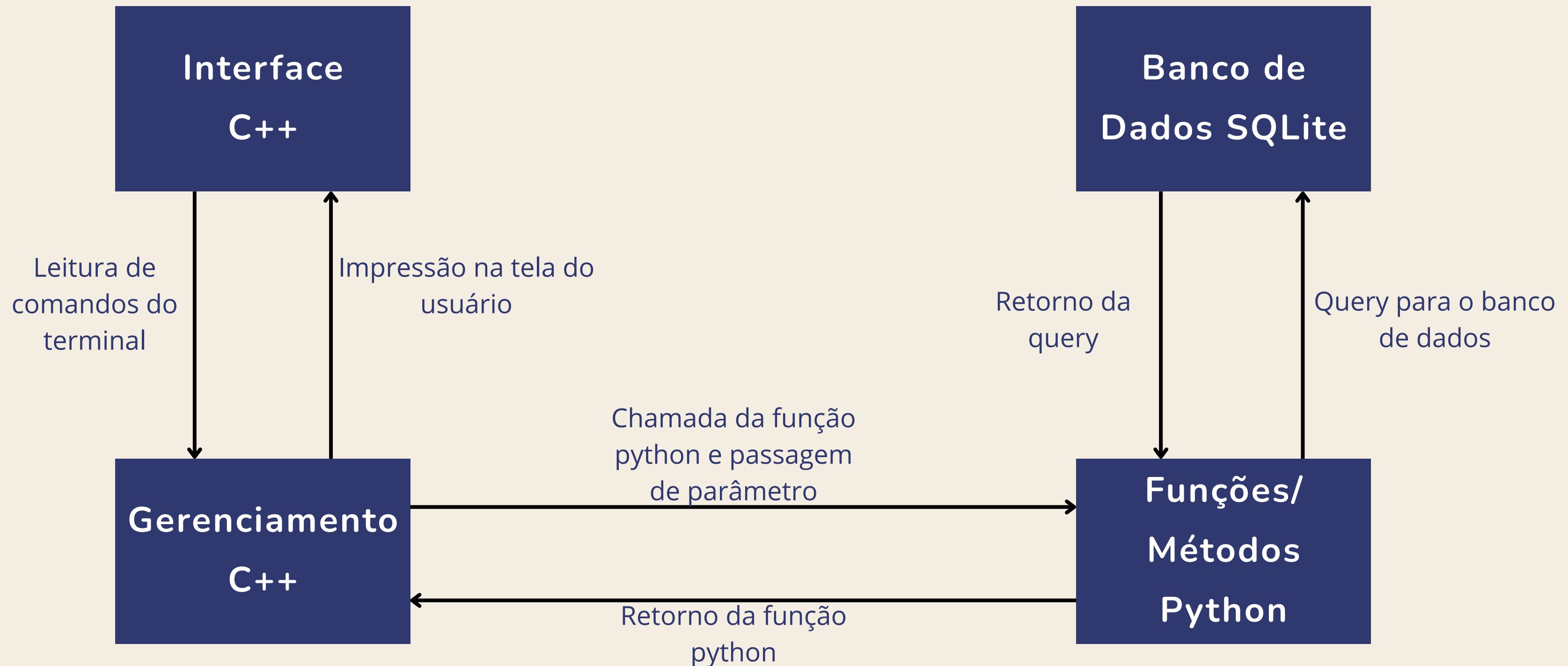
4. Análise de Código

PROBLEMA

Bibliotecas diariamente precisam lidar com uma quantidade massiva de livros, leitores e operações com esses dois (ex: um novo livro chegando a biblioteca ou um leitor querendo pegar um livro). Qual seria a melhor forma de fazer o gerenciamento desses livros?

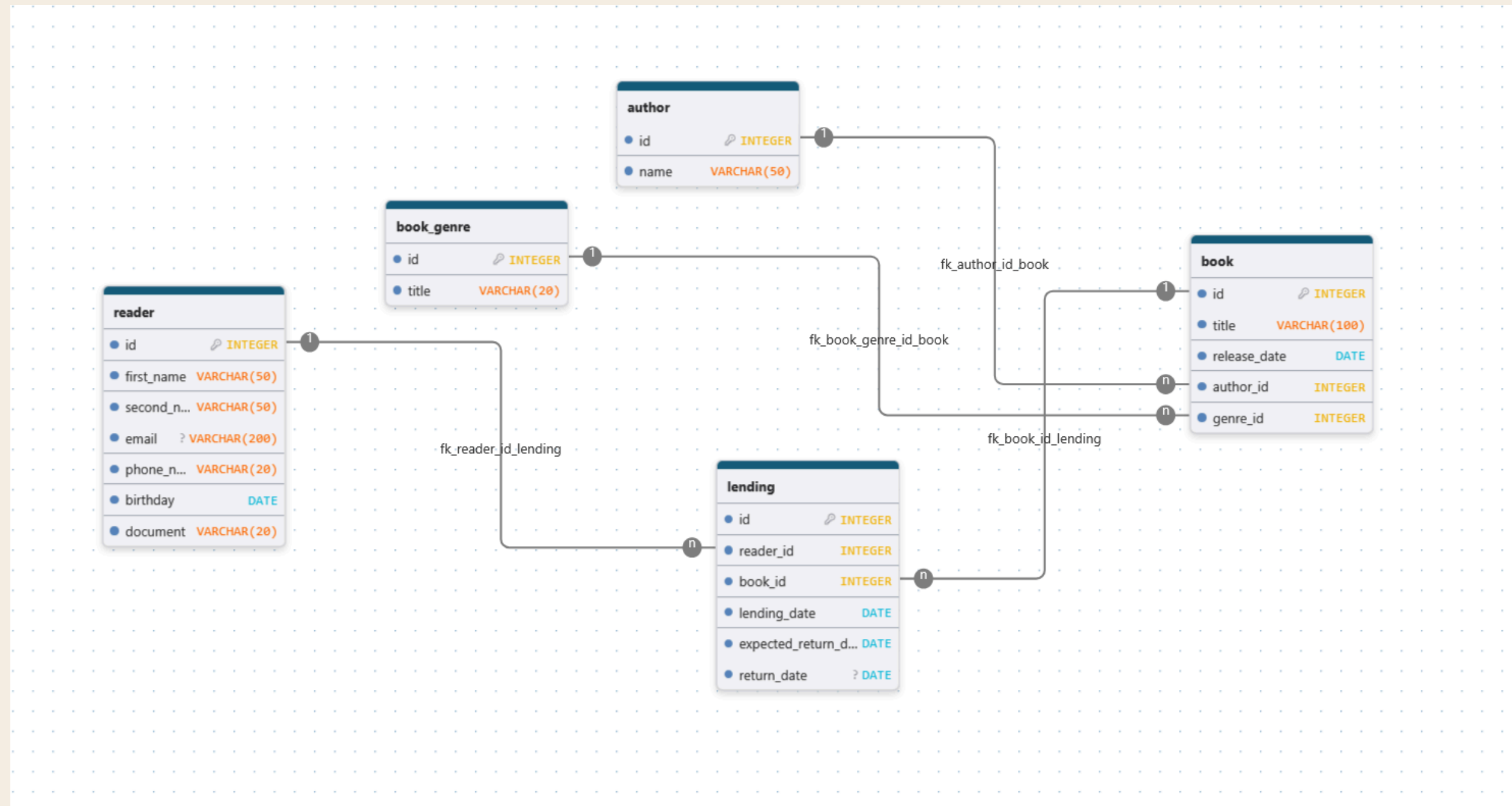
SOLUÇÃO

Para solucionar o problema, foi feito um programa que conta com a seguinte arquitetura



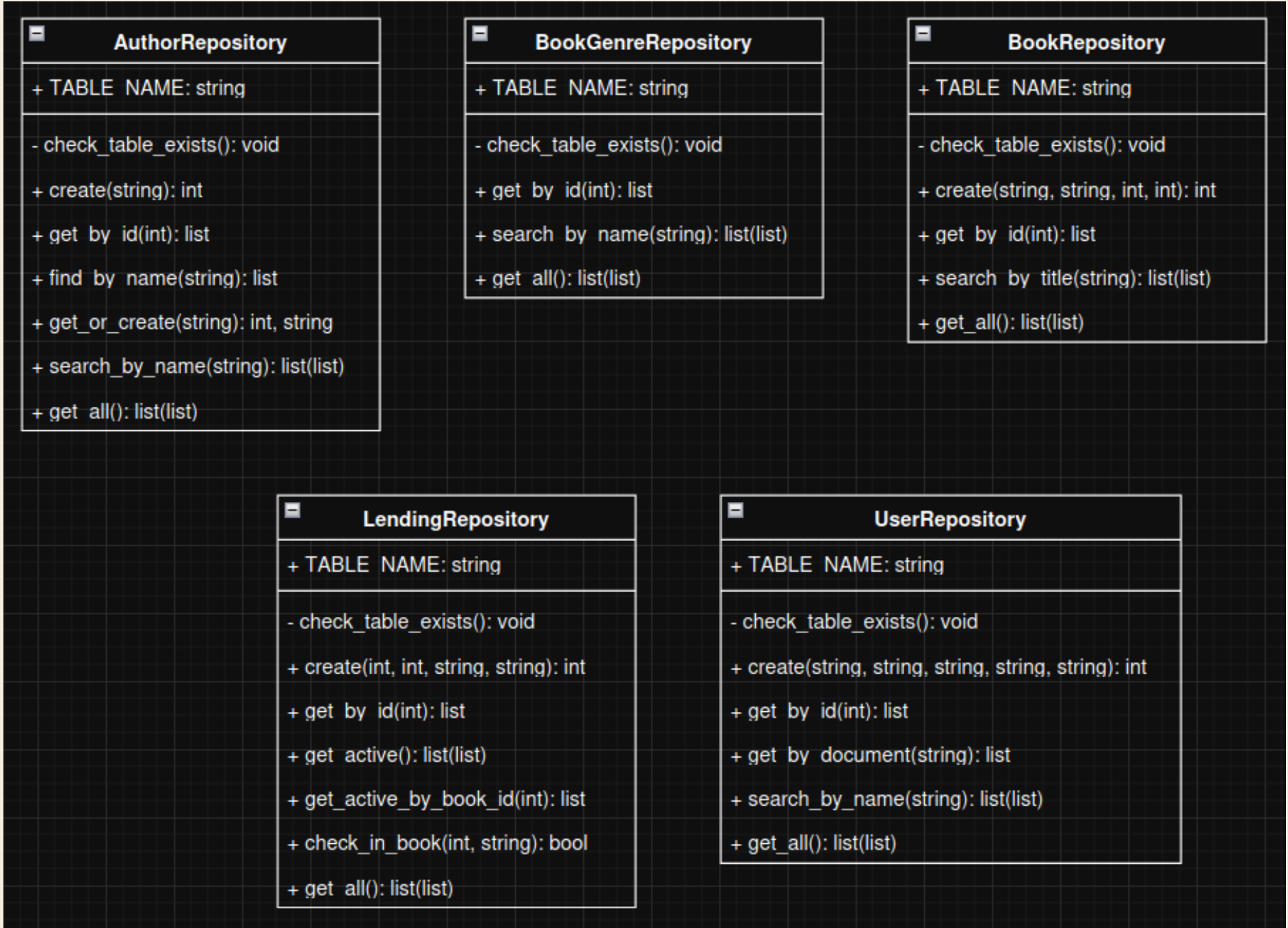
ARQUITETURA

Banco de Dados



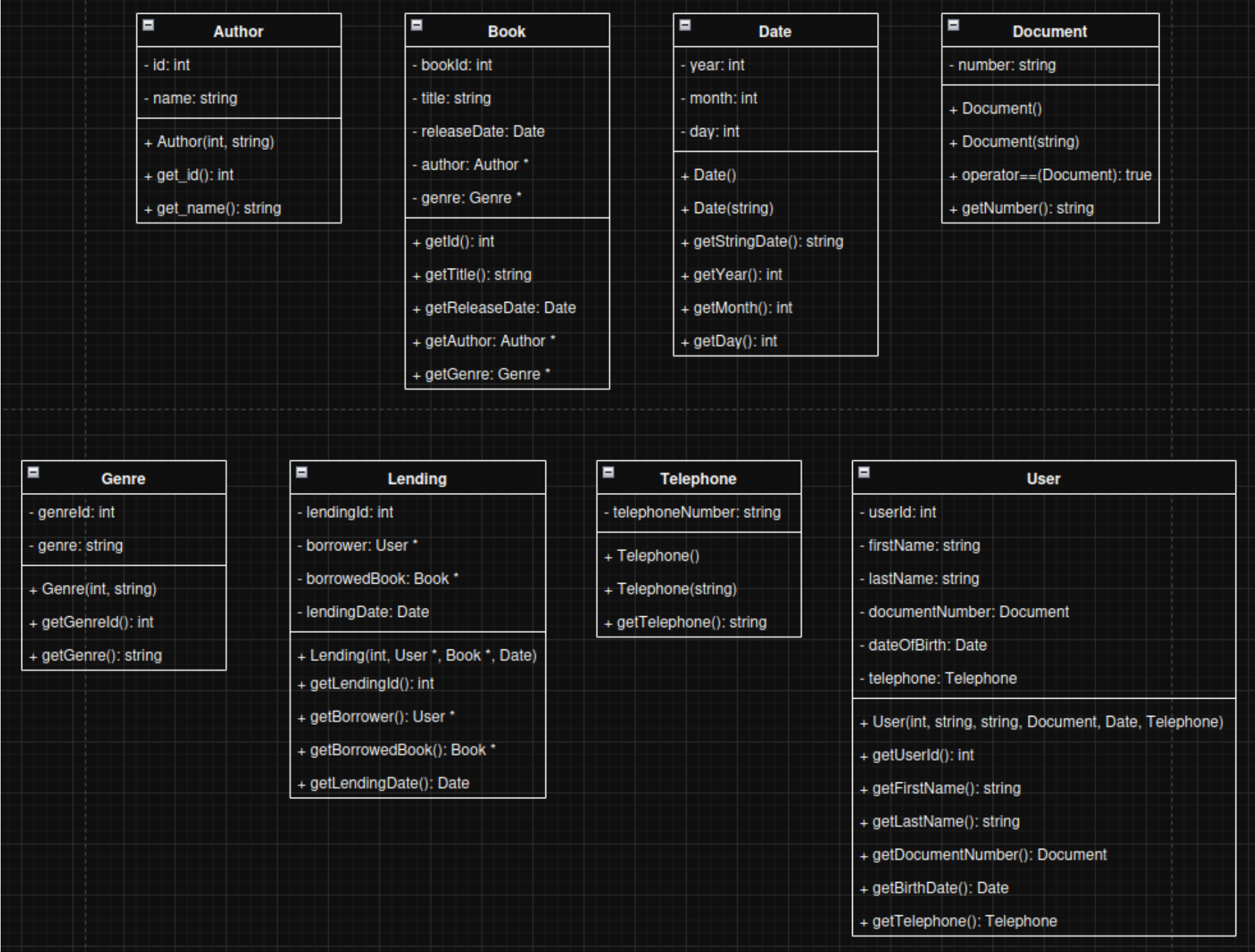
ARQUITERUA

Python



ARQUITERUA

C++



ARQUITERUA

Library: Orquestra os demais objetos

```
1 class Library {
2 public:
3     Library();
4
5     ~Library();
6
7     int newUser(std::string, std::string, Document, Date, Telephone);
8     void userInfo(Document);
9
10    int newBook(std::string, Date, std::string, int);
11
12    void bookInfo(int);
13
14    int landBook(Document, int);
15
16    void returnBook(Document, int);
17
18    void bookStatus(int);
19
20    void printUsers() const;
21
22    void printBooks() const;
23
24    // Get method
25    std::vector<const Genre *> getGenresVector() const;
```

```
1 private:
2     std::string dateStringFromNow(int daysFromNow = 0) const;
3     std::string normalizeDateString(std::string) const;
4     std::string setProper(std::string) const;
5     Author *findAuthorById(int) const;
6     Genre *findGenreById(int) const;
7     Genre *findGenreByName(const std::string &) const;
8
9     std::string libraryName;
10    std::vector<User *> users;
11    std::vector<Book *> books;
12    std::vector<Lending *> lendings;
13    std::vector<Author *> authors;
14    std::vector<Genre *> genres;
15    pybind11::scoped_interpreter interpreter;
16    pybind11::object authorRepository;
17    pybind11::object bookGenreRepository;
18    pybind11::object userRepository;
19    pybind11::object bookRepository;
20    pybind11::object lendingRepository;
21 };
```

PYBIND11

Exemplo: Livro

```
1 Library::Library() {
2     namespace py = pybind11;
3
4     // Import sys module from Python
5     py::module_ sys = py::module_::import("sys");
6
7     // Add pythonFunctions directory to the search path
8     sys.attr("path").attr("append")("./pythonFunctions");
9
10    // Import scripts and save as variables from library class
11    bookRepository =
12        py::module_::import("book_repository").attr(
13            "BookRepository")();
14    // [...]
15
16    // Load database to Library
17    // [...]
```

```
1 // book : vector of (id, title, release_date, author_id, genre_id)
2 auto bookList =
3     bookRepository.attr("get_all")()
4         .cast<std::vector<
5             std::tuple<int, std::string, std::string, int, int>>>
6         ();
7 for (const auto &book : bookList) {
8     int id = std::get<0>(book);
9     std::string title = setProper(std::get<1>(book));
10    Date releaseDate(normalizeDateString(std::get<2>(book)));
11    Author *author = findAuthorById(std::get<3>(book));
12    Genre *genre = findGenreById(std::get<4>(book));
13
14    if (author != nullptr && genre != nullptr) {
15        books.push_back(new Book
16            (id, title, releaseDate, author, genre));
17    }
```

SQLITE

Conexão com o banco de dados

```
1 import os
2 import sqlite3
3 from pathlib import Path
4
5
6 DEFAULT_DB_PATH = Path(__file__).resolve().parent.parent
  / "library.db"
7
8
9 def get_db_path():
10     return Path(os.environ.get("LIBRARY_DB_PATH",
11                                DEFAULT_DB_PATH))
12
13 def connect(db_path=None):
14     db_path = Path(db_path) if db_path is not None else
15     get_db_path()
16
17     if not db_path.exists():
18         raise FileNotFoundError(
19             f"Database not found at {db_path}
20             . Run scripts/create_database.py first."
21         )
22
23     conn = sqlite3.connect(db_path)
24     conn.row_factory = sqlite3.Row
25     conn.execute("PRAGMA foreign_keys = ON")
26     return conn
```

SQLITE

Exemplo: Livro

```
1 from db import connect
2
3
4 class BookRepository:
5     TABLE_NAME = "book"
6
7     def __init__(self, db_path=None):
8         self.db_path = db_path
9         self._check_table_exists()
10
11     def _check_table_exists(self):
12         with connect(self.db_path) as conn:
13             row = conn.execute(
14                 """
15                 SELECT name
16                 FROM sqlite_master
17                 WHERE type = 'table'
18                 AND name = ?
19                 """,
20                 (self.TABLE_NAME,),
21             ).fetchone()
22
23         if row is None:
24             raise RuntimeError(
25                 f"Table '{self.TABLE_NAME}' does not exist. "
26                 "Run scripts/create_database.py first."
27             )
28
```

```
1     def create(self, title, release_date, author_id, genre_id):
2         with connect(self.db_path) as conn:
3             cursor = conn.execute(
4                 f"""
5                 INSERT INTO {self.TABLE_NAME}
6                 (title, release_date, author_id, genre_id)
7                 VALUES (?, ?, ?, ?)
8                 """,
9                 (title, release_date, author_id, genre_id),
10            )
11            conn.commit()
12            return cursor.lastrowid
13
14     def get_all(self):
15         with connect(self.db_path) as conn:
16             rows = conn.execute(
17                 f"""
18                 SELECT id, title, release_date, author_id, genre_
19                 id
20                 FROM {self.TABLE_NAME}
21                 ORDER BY title
22                 """
23             ).fetchall()
24             return [tuple(row) for row in rows]
```

COMO RODAR?

Makefile

How to Run

1. Setup

On Ubuntu/Debian, run this once to install C++ compiler, SQLite CLI, Python venv support, and Python development headers required by pybind11.

```
make system-deps
```

And this to creates `.venv/` and installs `requirements.txt` , including `pytest` and `pybind11` .

```
make setup
```

2. Create the database

To create the database with sample authors, books, readers, and lendings:

```
make data  
make sampledata
```

3. Run program

Build and run main program

```
make run
```

OBRIGADO

Rafael Gabriel - 121087555

Luiza Pacheco - 119169147